common-library: code explained

banking-platform project ka ab tak likha hua code, line-by-line, simple Hinglish mein.

Folder: `common-library/src/main/java/com/banking/common/` · 8 files · 31 Oct 2026 tak ka code

> **Yeh library kya hai?** Aapke project mein 6 services hain (account, transaction, fraud, notification, audit, gateway). `common-library` woh shared code hai jo **saari services** use karengi, taaki errors, API ka jawab aur Kafka events har jagah ek jaise rahein.

Group	Files	Kaam
`exception/`	ErrorCode, BankingException	Errors sambhalna
`dto/`	ApiResponse, ErrorResponse, PagedResponse	API ka jawab kaisa dikhega
`event/`	EventType, BaseEvent	Kafka events ki neev
`util/`	DateTimeUtil	Date/time helpers (adhoori, typo hai)

Group 1: Errors sambhalna

1. ErrorCode.java done `exception/ErrorCode.java`

Saare errors ki ek fixed list, aur har error ka HTTP status.

```java
package com.banking.common.exception;

import lombok.Getter;
import org.springframework.http.HttpStatus;

@Getter
public enum ErrorCode {

    // Generic
    INTERNAL_ERROR(HttpStatus.INTERNAL_SERVER_ERROR),
    VALIDATION_ERROR(HttpStatus.BAD_REQUEST),
    RESOURCE_NOT_FOUND(HttpStatus.NOT_FOUND),
    UNAUTHORIZED(HttpStatus.UNAUTHORIZED),
    FORBIDDEN(HttpStatus.FORBIDDEN),

    // Account
    ACCOUNT_NOT_FOUND(HttpStatus.NOT_FOUND),
    ACCOUNT_FROZEN(HttpStatus.CONFLICT),
    ACCOUNT_CLOSED(HttpStatus.CONFLICT),
    DUPLICATE_ACCOUNT(HttpStatus.CONFLICT),
    INSUFFICIENT_FUNDS(HttpStatus.CONFLICT),

    // Transaction
    TRANSACTION_NOT_FOUND(HttpStatus.NOT_FOUND),
    TRANSACTION_FAILED(HttpStatus.UNPROCESSABLE_ENTITY),
    DUPLICATE_TRANSACTION(HttpStatus.CONFLICT),
    LIMIT_EXCEEDED(HttpStatus.CONFLICT),

    // Fraud
```

```
30      FRAUD_ALERT_NOT_FOUND(HttpStatus.NOT_FOUND),
31      FRAUD_BLOCKED(HttpStatus.FORBIDDEN),
32
33      // Notification
34      NOTIFICATION_DELIVERY_FAILED(HttpStatus.BAD_GATEWAY),
35
36      // Gateway
37      SERVICE_UNAVAILABLE(HttpStatus.SERVICE_UNAVAILABLE);
38
39      private final HttpStatus status;
40
41      ErrorCode(HttpStatus status) {
42          this.status = status;
43      }
44  }
```

- **Line 7,** `enum`**:** fixed constants ki list. Isme kul 18 errors hain, aur inke alawa koi naya ErrorCode ban hi nahi sakta.
- **Line 10–37:** har error ke saath HTTP status juda hai. `ACCOUNT_NOT_FOUND(HttpStatus.NOT_FOUND)` matlab account na mile toh client ko 404 bhejo. 409 CONFLICT matlab request sahi hai lekin account ki halat se takraati hai, jaise account frozen ho ya balance kam ho.
- **Line 37:** sirf aakhri constant ke baad `;` aata hai. Isse Java samajhta hai ki list khatam ho gayi.
- **Line 39–43:** bracket wali value constructor mein jaati hai aur `status` field mein save hoti hai. `final` ka matlab hai ki ek baar set hone ke baad badlegi nahi.
- **Line 6,** `@Getter` **(Lombok):** apne aap `getStatus()` method bana deta hai.

> **Fayda:** kis error par kaunsa HTTP status jaayega, yeh poore project mein ek hi jagah tay hota hai.

2. BankingException.java done `exception/BankingException.java`

Aapka apna exception. Service mein kuch galat ho toh yahi throw hoga.

```
1  package com.banking.common.exception;
2
3  import lombok.Getter;
4  import org.springframework.http.HttpStatus;
5
6  @Getter
7  public class BankingException extends RuntimeException {
8
9      private final ErrorCode errorCode;
10
11     public BankingException(ErrorCode errorCode, String message) {
12         super(message);
13         this.errorCode = errorCode;
14     }
15
16     public BankingException(ErrorCode errorCode, String message, Throwable cause) {
17         super(message, cause);
18         this.errorCode = errorCode;
19     }
20
21     public BankingException(String message) {
22         this(ErrorCode.INTERNAL_ERROR, message);
23     }
```

```
24
25      public HttpStatus getStatus() {
26          return errorCode.getStatus();
27      }
28  }
```

- **Line 7,** `extends RuntimeException`**:** unchecked exception hai, isliye har method par `throws` ya try-catch likhna zaroori nahi. Spring ka `@Transactional` bhi default rollback isi type par karta hai.
- **Line 9:** har exception ke andar ek `ErrorCode` hota hai, jisse pata chalta hai ki kaunsa HTTP status bhejna hai.
- **Line 11–14:** normal constructor (code + message). `super(message)` message ko parent class ko de deta hai.
- **Line 16–19:** code + message + `cause`. Jab kisi doosre error (jaise database error) ko wrap karna ho, tab original error bhi saath rehta hai.
- **Line 21–23:** sirf message. `this(...)` upar wala constructor call karta hai, default `INTERNAL_ERROR` ke saath.
- **Line 25–27:** `getStatus()` exception se seedha HTTP status de deta hai.

Aage service mein aise use hoga:

```
throw new BankingException(ErrorCode.ACCOUNT_NOT_FOUND, "Account ACC-1001 nahi mila");
```

Group 2: API ka jawab kaisa dikhega

3. ApiResponse.java done dto/ApiResponse.java

Har API ka jawab isi wrapper mein jaayega: ya `data`, ya `error`.

```
1  package com.banking.common.dto;
2
3  import com.fasterxml.jackson.annotation.JsonInclude;
4  import lombok.Getter;
5
6  @Getter
7  @JsonInclude(JsonInclude.Include.NON_NULL)
8  public class ApiResponse<T> {
9
10     private final T data;
11
12     private final ErrorResponse error;
13
14     public ApiResponse(T data) {
15         this.data = data;
16         this.error = null;
17     }
18
19     public ApiResponse(ErrorResponse error) {
20         this.data = null;
21         this.error = error;
22     }
23 }
```

- **Line 8, `<T>` (generics):** T ek placeholder type hai. `data` kisi bhi type ka ho sakta hai: account, transaction, list, kuch bhi.
- **Line 7, `@JsonInclude(NON_NULL)`:** jo field null hai woh JSON mein dikhegi hi nahi.
- **Line 14–17:** success wala constructor, isme `error` null rehta hai.
- **Line 19–22:** error wala constructor, isme `data` null rehta hai. Java argument ka type dekh kar sahi constructor chunta hai.

Success par client ko milega:

```
{ "data": { "accountNumber": "ACC-1001", "balance": 25000 } }
```

4. ErrorResponse.java done dto/ErrorResponse.java

Error ka JSON shape: message, code aur status.

```
1  package com.banking.common.dto;
2
3  import com.banking.common.exception.ErrorCode;
4  import lombok.Getter;
5
6  public class ErrorResponse {
7
8      @Getter
9      private final String message;
```

```
10
11      @Getter
12      private final String code;
13
14      @Getter
15      private final int status;
16
17      public ErrorResponse(ErrorCode errorCode, String message) {
18          this.code = errorCode.name();
19          this.message = message;
20          this.status = errorCode.getStatus().value();
21      }
22
23  }
```

- **Line 8–15:** teen fields. `message` insaan ke padhne ke liye, `code` program ke liye, `status` HTTP number.
- **Line 18, `name()`:** enum constant ka naam string mein deta hai, jaise `"ACCOUNT_NOT_FOUND"`.
- **Line 20, `getStatus().value()`:** HttpStatus se number deta hai, jaise `404`.
- Aap sirf ErrorCode aur message doge, baaki fields yeh class khud bhar degi.

Error par client ko milega:

```
{ "error": { "message": "Account ACC-1001 nahi mila", "code": "ACCOUNT_NOT_FOUND", "status": 404 } }
```

5. PagedResponse.java done `dto/PagedResponse.java`

Lambi lists ko pages mein bhejne ke liye, jaise 1000 transactions ki jagah ek baar mein 20.

```
1  package com.banking.common.dto;
2
3  import lombok.Getter;
4  import lombok.Builder;
5  import java.util.List;
6
7  @Getter
8  @Builder
9  public class PagedResponse<T> {
10
11      private final List<T> data;
12
13      private final int page;
14
15      private final int size;
16
17      private final long totalElements;
18
19      private final int totalPages;
20
21      private final boolean last;
22 }
```

Field	Matlab
`data`	is page ke items
`page`	kaunsa page (0 se shuru)
`size`	ek page mein kitne items

`totalElements`	total kitne records hain
`totalPages`	total kitne pages hain
`last`	kya yeh aakhri page hai

Line 8, `@Builder` **:** object aise banta hai: `PagedResponse.builder().data(list).page(0).size(20).build()`

Group 3: Kafka events

6. EventType.java commit baaki `event/EventType.java`

System ke saare events ki list, aur har event kis Kafka topic par jaayega.

```java
package com.banking.common.event;

import lombok.Getter;

@Getter
public enum EventType{
    //Account
    ACCOUNT_CREATED(Topics.ACCOUNT),
    ACCOUNT_STATUS_CHANGED(Topics.ACCOUNT),

    //Transaction
    TRANSACTION_INITIATED(Topics.TRANSACTION),
    TRANSACTION_COMPLETED(Topics.TRANSACTION),
    TRANSACTION_FAILED(Topics.TRANSACTION),

    //fraud
    FRAUD_DETECTED(Topics.FRAUD),
    FRAUD_CLEARED(Topics.FRAUD),

    // Notification
    NOTIFICATION_REQUESTED(Topics.NOTIFICATION);

    private final String topic;

    EventType(String topic) {
        this.topic = topic;
    }

    public static class Topics {
        public static final String ACCOUNT = "banking.account.events";
        public static final String TRANSACTION = "banking.transaction.events";
        public static final String FRAUD = "banking.fraud.events";
        public static final String NOTIFICATION = "banking.notification.events";

        private Topics() {
            // Private constructor to prevent instantiation
        }
    }

}
```

- **Line 8–21:** 8 tarah ke events, jaise "account bana" ya "transaction shuru hua". Har event ka ek **Kafka topic** hai, yaani woh channel jahan event bheja jaayega.
- **Line 23–27:** ErrorCode jaisa hi pattern hai, bas yahan value topic ka naam (String) hai.
- **Line 29–33,** `Topics`**:** andar wali `static` class jisme sirf topic ke naam hain. Use karne ke liye object banane ki zaroorat nahi: `EventType.Topics.ACCOUNT`.
- **Line 35–37:** private constructor, taaki koi `new Topics()` na bana sake.

7. BaseEvent.java commit baaki event/BaseEvent.java

Saare events ka parent class. Har event ke common fields yahan hain.

```java
package com.banking.common.event;

import com.fasterxml.jackson.annotation.JsonIgnore;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.Instant;
import java.util.UUID;

@Getter
@Setter
@NoArgsConstructor
public abstract class BaseEvent {

    public static final int CURRENT_SCHEMA_VERSION = 1;

    private String eventId;

    private EventType eventType;

    private String aggregateId;

    private Instant occurredAt;

    private String correlationId;

    private int schemaVersion;

    protected BaseEvent(EventType eventType, String aggregateId, String correlationId) {
        this.eventId = UUID.randomUUID().toString();
        this.eventType = eventType;
        this.aggregateId = aggregateId;
        this.correlationId = correlationId;
        this.occurredAt = Instant.now();
        this.schemaVersion = CURRENT_SCHEMA_VERSION;
    }

    @JsonIgnore
    public String getTopic() {
        return eventType.getTopic();
    }
}
```

Field	Matlab
`eventId`	har event ka unique id (duplicate pakadne ke liye)
`eventType`	kaunsa event hai
`aggregateId`	kis cheez ka event, jaise accountId
`occurredAt`	kab hua (UTC time)
`correlationId`	ek transfer ke saare events ko jodta hai

`schemaVersion`	event format ka version, abhi 1

- **Line 14,** `abstract`**:** iska seedha object nahi banega. Aage `AccountCreatedEvent extends BaseEvent` jaisi classes banengi.
- **Line 30–37, constructor:** aap sirf 3 cheezein doge: type, aggregateId aur correlationId. `eventId` (`UUID.randomUUID()`) aur time (`Instant.now()`) apne aap set ho jaate hain. `protected` ka matlab hai ki sirf child classes ise call kar sakti hain.
- **Line 11–13,** `@Setter` **+** `@NoArgsConstructor`**:** Kafka se aaya JSON wapas Java object banane ke liye Jackson ko khaali constructor aur setters chahiye.
- **Line 39–42,** `getTopic()`**:** batata hai ki event kis topic par bhejna hai. `@JsonIgnore` ki wajah se topic JSON ke andar nahi jaata.

Group 4: Adhoori file

8. DateTimeUtil.java

build todega `util/DateTimeUtil.java`

```java
package com.banking.common.util;

import java.time.Instant;
import java.time.LocalDateTime;
import java.time.Duaration;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;

public final class DateTimeUtil {


}
```

Line 5 par typo: `Duaration` naam ki koi class Java mein nahi hai, isliye poora common-library build fail hoga. Sahi hai `import java.time.Duration;`. Class abhi khaali hai; isme aage date/time ke helper methods aayenge.

Sab ek saath kaise judte hain

```
Service mein galti     →  throw new BankingException(ErrorCode.ACCOUNT_NOT_FOUND, "...")
                       →  ErrorResponse banta hai  (code, message, 404)
                       →  ApiResponse mein lipat ke JSON client ko jaata hai

Service mein kaam hua  →  BaseEvent wala event banta hai  (EventType se)
                       →  getTopic() wale Kafka topic par chala jaata hai
```

Agla kaam: DateTimeUtil ka typo fix karo, phir `mvn -pl common-library clean install` se build check karo, aur EventType/BaseEvent ko commit kar do.